

Adder:

#6

Day MTWTFSS

Adder:

→ In electronics an adder is digital circuit that perform addition of numbers.

→ In modern computers adder reside in the arithmetic logic unit (ALU).

→ Adders are important not only in the computers but also in many types of digital systems in which the numeric data are processed.

Types of adder:

- Half adder
- Full adder

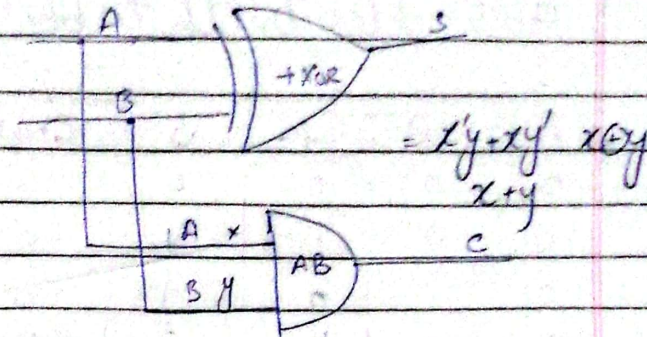
Half adder:

→ The half adder accepts two binary digits on its inputs and produces two binary digits outputs, a sum bit and a carry bit.

→ The half adder is an example of a simple, functional digital circuit built from two logic gates.

→ The half adder adds to one-bit binary numbers. The output is the sum of the two bits and the carry.

A		Σ		A ⊕ B sum	
B		Σ		A · B carry	
A	B	S	Cont	Truth table	
0	0	0	0	Sum	
0	1	1	0	$x'y + xy'$	
1	0	1	0		
1	1	0	1	$C = xy$	



Full adder:

→ The full adder accepts two inputs bits and an input carry and generates a sum output and an output carry.

→ The full adder circuit adds three one-bit binary numbers (C_{in}, A, B) and outputs two one-bit binary numbers, a sum and a carry out.

→ The full-adder is usually a component in a cascade of adders, which add 8, 16, 32 etc. binary numbers.

A	Σ	Σ	Sum
B			
Carry in			Carry out

Date _____

Day MTWTF

Half adder:

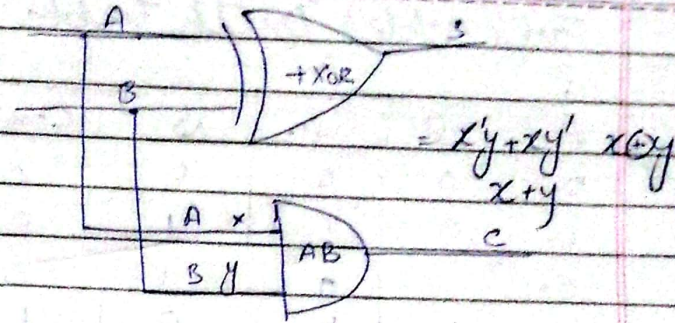
→ The half adder accepts two binary digits on its inputs and produce two binary digits outputs, a sum bit and a carry bit.

→ The half adder is an example of a simple, functional digital circuit built from two logic gates.

→ The half adder adds to one-bit binary numbers. The output is the sum of the two bits and the carry.

A	B	Σ	Σ	$A \oplus B$ sum
			Carry	$A \cdot B$ carry
		Σ	Carry	
0	0	0	0	Sum
0	1	1	0	$x'y + xy'$
1	0	1	0	
1	1	0	1	$C = xy$

Date _____

Day MTWTF

Full adder:

→ The full adder accepts two inputs bits and an input carry and generates a sum output and an output carry.

→ The full adder circuit adds three one-bit binary numbers (C_{in}, A, B) and outputs two one-bit binary numbers, a sum and a carry out.

→ The full-adder is usually a component in a cascade of adders, which add 8, 16, 32 etc. binary numbers.

A	B	Σ	Σ	Sum
				Carry out

Full adder truth table

A	B	Cin	S	Count
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S = (A \oplus B) \oplus cin$$

$$S = \bar{A}\bar{B}cin + \bar{A}B\bar{c}in + A\bar{B}\bar{c}in + ABcin$$

$$C = AB + cin(A \oplus B)$$

$$C = ABcin + \bar{A}Bcin + A\bar{B}cin + ABcin$$

Sum:

	$\bar{B}cin$	$\bar{B}\bar{c}$	$\bar{B}c$	BC	BC
A	00	01	11	10	
0	0	1	0	1	
1	1	0	1	0	

$= \bar{A}\bar{B}cin + \bar{A}B\bar{c}in + A\bar{B}\bar{c}in + ABcin$
 $= \bar{A}cin + Bcin + A\bar{B}cin + ABcin$
 $= \bar{A}cin + Bcin + cin(A \oplus B)$
 $= cin(\bar{A} + B + A \oplus B)$
 $= cin(\bar{A} + B + A \oplus B)$
 $= cin(\bar{A} + B + A \oplus B)$

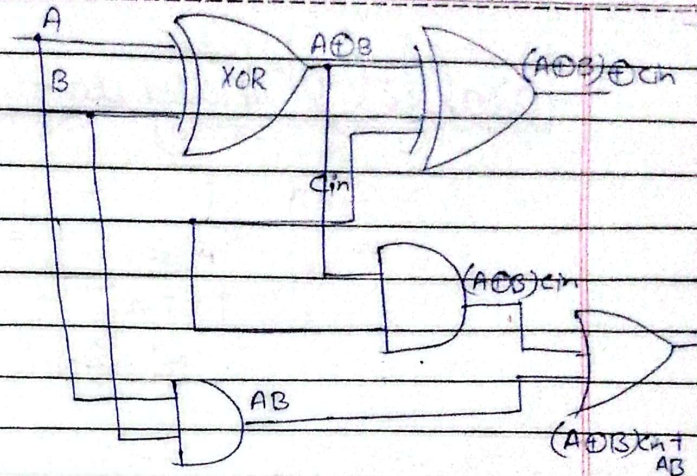
→ Such configuration is called check bold configuration.

Carry:

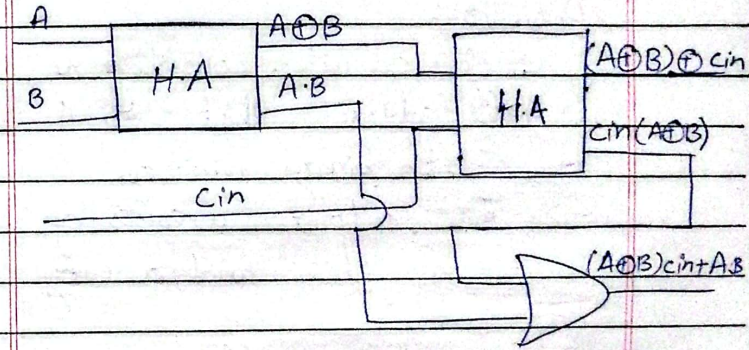
	$\bar{B}cin$	$\bar{B}\bar{c}$	$\bar{B}c$	BC	BC
A	00	01	11	10	
0					
1					

$$Count = Bcin + Acin + AB$$

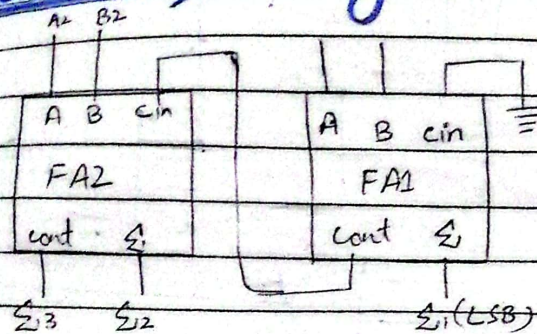
$$= (\bar{A} + B)cin + AB$$



Implemented of full adder using half adder



Parallel Binary adders



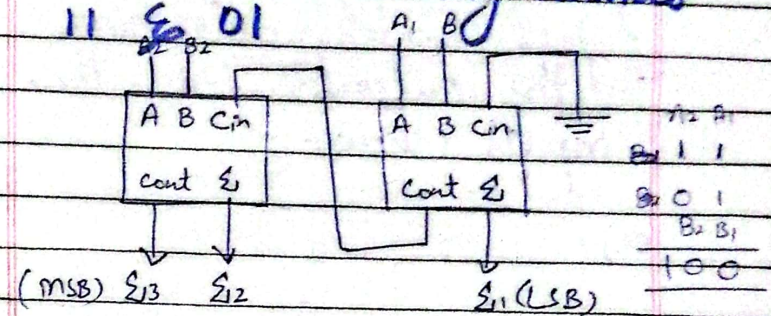
$$\begin{array}{r} A_2 A_1 \\ + B_2 B_1 \\ \hline \Sigma_3 \Sigma_2 \Sigma_1 \end{array}$$

Definition:

→ A circuit, consisting of n full adders, that will add n -bit binary numbers.
 → The output consists of n sum bits and a carry bit.

→ Cout of one full adder is connected to cin of the next full adder.

Add 2-bit binary numbers



$$\begin{array}{l} = (A \oplus B) \oplus C_{in} \\ = (1 \oplus 1) \oplus 0 \\ = 0 \oplus 0 \\ = 0 \\ \Sigma_1 = 0 \end{array} \quad \begin{array}{l} = (A \oplus B) C_{in} + A \cdot B \\ = (1 \oplus 1) \cdot 0 + 1 \cdot 1 \\ = 0 + 1 \\ = 1 \\ C_{out} = 1 \end{array}$$

$$\begin{array}{l} = (A \oplus B) \oplus C_{in} \\ = (1 \oplus 0) \oplus 1 \\ = 1 \oplus 1 \\ = 0 \\ \Sigma_2 = 0 \end{array} \quad \begin{array}{l} = (A \oplus B) C_{in} + A \cdot B \\ = (1 \oplus 0) \cdot 1 + 1 \cdot 0 \\ = 1 \cdot 1 + 0 \\ = 1 + 0 \\ = 1 \\ \Sigma_3 = 1 \end{array}$$

$$\begin{array}{l} = \Sigma_3 \Sigma_2 \Sigma_1 \\ = 100 \text{ Ans} \end{array}$$

Date _____

Day MTWTF

for each of the three full adders determine the outputs for the inputs shown.

a)	A	Σ	1
	B	Σ	
	cin	cout	0

$$\begin{aligned}\Sigma_1 &= (A \oplus B) \oplus \text{cin} \\ &= (1 \oplus 0) \oplus 0 \\ &= 1 \oplus 0\end{aligned}$$

$$\Sigma_1 = 1$$

$$\begin{aligned}\text{cout} &= (A \oplus B) \text{cin} + AB \\ &= (1 \oplus 0) 0 + 1 \cdot 0 \\ &= 1 \cdot 0 + 0 \\ &= 0 + 0\end{aligned}$$

$$\text{cout} = 0$$

b)	A	Σ	0
	B	Σ	
	cin	cout	1

$$\begin{aligned}\Sigma_1 &= (A \oplus B) \oplus \text{cin} \\ &= (1 \oplus 1) \oplus 0 \\ &= 0 \oplus 0 \\ &= 0\end{aligned}$$

$$\begin{aligned}\text{cout} &= (A \oplus B) \text{cin} + AB \\ &= (1 \oplus 1) 0 + 1 \cdot 1 \\ &= 0 + 1 \\ &= 1\end{aligned}$$

Date _____

Day MTWTF

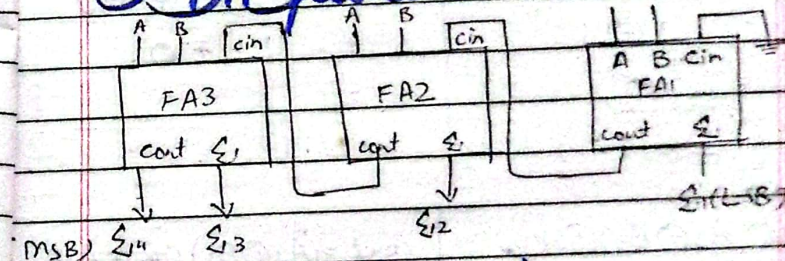
A full adder has cin=1. What are the sum (Σ) and the output carry (cout) when A=1 & B=1?

A	Σ	Σ	1
B		cout	1
cin			

$$\begin{aligned}\Sigma_1 &= (A \oplus B) \oplus \text{cin} \\ &= (1 \oplus 1) \oplus 1 \\ &= 0 \oplus 1 \\ \Sigma_1 &= 1\end{aligned}$$

$$\begin{aligned}\text{cout} &= (A \oplus B) \text{cin} + AB \\ &= (1 \oplus 1) 1 + 1 \\ &= 0 + 1 \\ &= 1\end{aligned}$$

3-bit parallel adder:

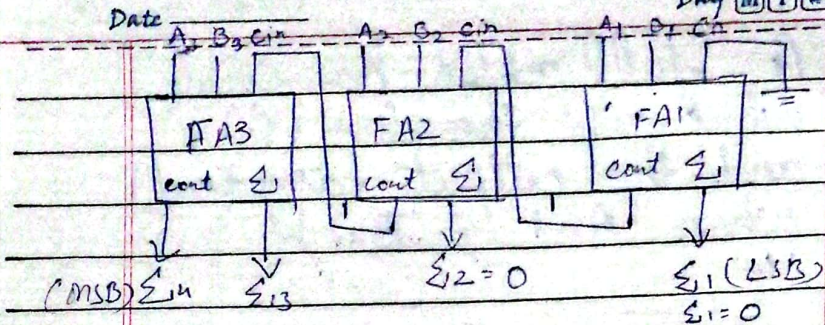


Add 3-bit numbers:

101 & 011

A ₃	A ₂	A ₁	B ₃	B ₂	B ₁
1	0	1	0	1	1
0	1	0	1	0	0
1	0	0			

Date _____

Day MTWTF

$$\Sigma_1 = (A_1 \oplus B_1) \oplus C_{in}$$

$$= (1 \oplus 1) \oplus 0$$

$$= 0 \oplus 0$$

$$\Sigma_1 = 0$$

$$C_{out} = (A \oplus B) C_{in} + AB_1$$

$$= (1 \oplus 1) 0 + 1 \cdot 1$$

$$= 0 + 1$$

$$= 1$$

$$\Sigma_2 = (A_2 \oplus B_2) \oplus C_{in}$$

$$= (0 \oplus 1) \oplus 1$$

$$= 1 \oplus 1$$

$$\Sigma_2 = 0$$

$$C_{out} = (A \oplus B_2) C_{in} + A_2 B_2$$

$$= (0 \oplus 1) 1 + 1 \cdot 0$$

$$= 1 + 0$$

$$= 1$$

$$\Sigma_3 = (A_3 \oplus B_3) \oplus C_{in}$$

$$= (1 \oplus 0) \oplus 1$$

$$= 1 \oplus 1$$

$$\Sigma_3 = 0$$

$$C_{out} = (A_3 \oplus B_3) C_{in} + A_3 B_3$$

$$= (1 \oplus 0) 1 + 1 \cdot 0$$

$$= 1 + 0$$

$$= 1$$

$$\Sigma_4 = 1$$

$$\Sigma_4 \Sigma_3 \Sigma_2 \Sigma_1 = 1000 \quad \text{Ans}$$

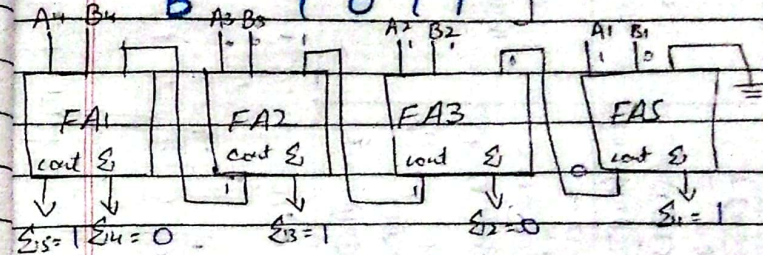
Date _____

Day MTWTF

4-bit Binary

$$A \quad 1010 \quad \Sigma_4 \Sigma_3 \Sigma_2 \Sigma_1 = 1010$$

$$B \quad 1011$$



$$\Sigma_1 = (A_1 \oplus B_1) \oplus C_{in}$$

$$= (0 \oplus 1) \oplus 0$$

$$= 1 \oplus 0$$

$$\Sigma_1 = 1$$

$$C_{out} = (A \oplus B) C_{in} + AB_1$$

$$= (0 \oplus 1) 0 + 0 \cdot 1$$

$$= 0 \oplus 0$$

$$= 0$$

$$\Sigma_2 = (A_2 \oplus B_2) \oplus C_{in}$$

$$= (1 \oplus 1) \oplus 0$$

$$= 0 \oplus 0$$

$$\Sigma_2 = 0$$

$$C_{out} = (A_2 \oplus B_2) C_{in} + A_2 B_2$$

$$= (1 \oplus 1) 0 + 1 \cdot 1$$

$$= 0 + 1$$

$$= 1$$

$$\Sigma_3 = (A_3 \oplus B_3) \oplus C_{in}$$

$$= (0 \oplus 0) \oplus 1$$

$$= 0 \oplus 1 = 1$$

$$\Sigma_3 = 1$$

$$C_{out} = (A_3 \oplus B_3) C_{in} + A_3 B_3$$

$$= (0 \oplus 0) 1 + 0 \cdot 0$$

$$= 0 + 0 = 0$$

$$\Sigma_4 = (A_4 \oplus B_4) \oplus C_{in}$$

$$= (1 \oplus 1) \oplus 0$$

$$= 0 \oplus 0 = 0$$

$$C_{out} = (A_4 \oplus B_4) C_{in} + A_4 B_4$$

$$= (1 \oplus 1) 0 + 1 \cdot 1$$

$$= 0 + 1 = 1$$

$$\Sigma_4 = 1$$

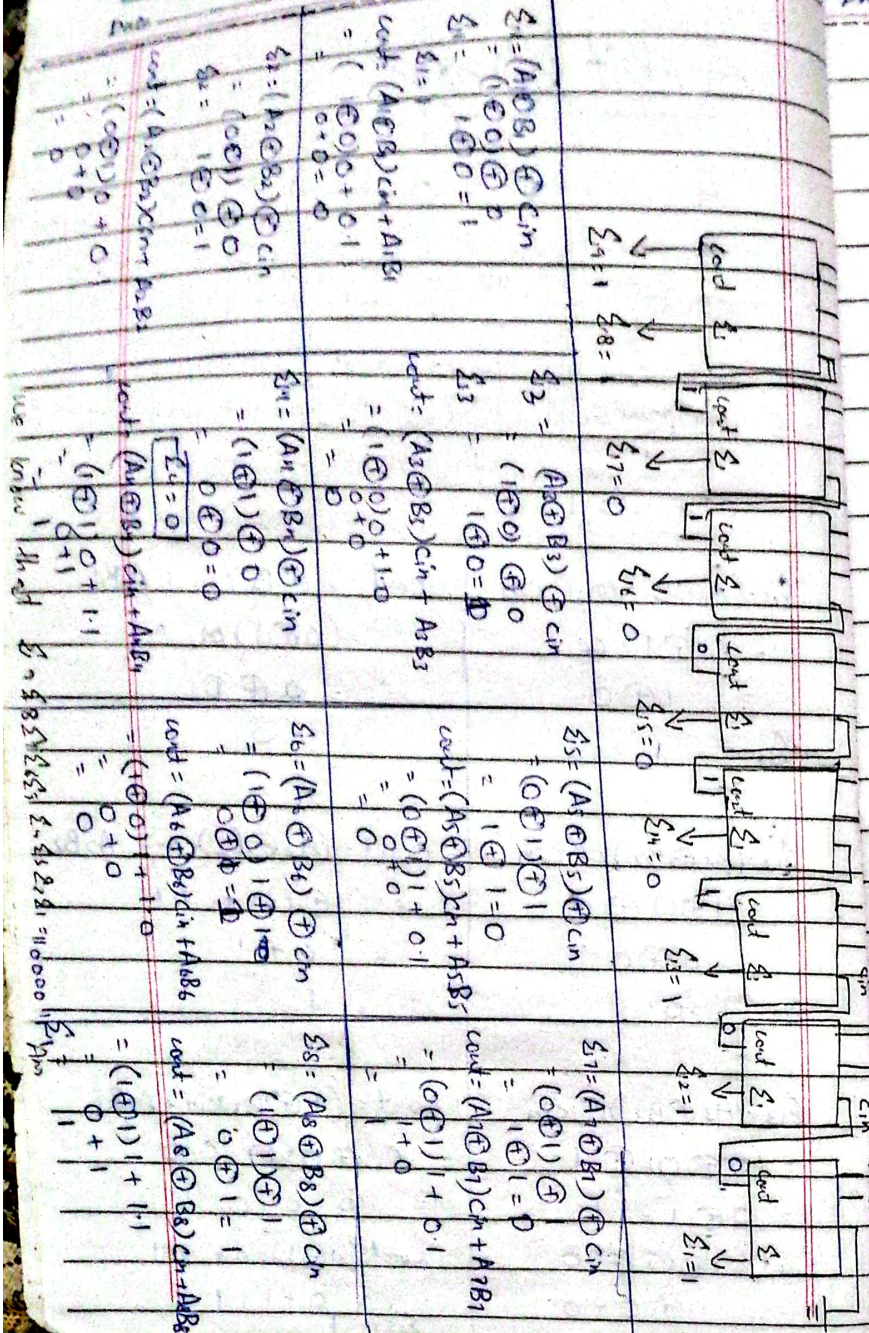
8-Bit:

A 1010 1101
B 1101 1010
Cin 1000 0111

Day M T W T F S S

Date

Day M T W T F S S



Ripple Carry & Look Ahead Carry

→ Application of full adder.
→ The ripple carry adder is constructed by cascading full adder blocks in series.

→ The carryout of one stage is fed directly to the carry-in of the next stage.

→ This adder is called a ripple carry adder because each carry bit gets rippled into the next stage.

→ for an n-bit ripple adder, we require n full adders.

→ Propagation delays inside the logic circuitry is the reason behind this.

→ Propagation delay is time elapsed between the application of an input and occurrence of the corresponding output.

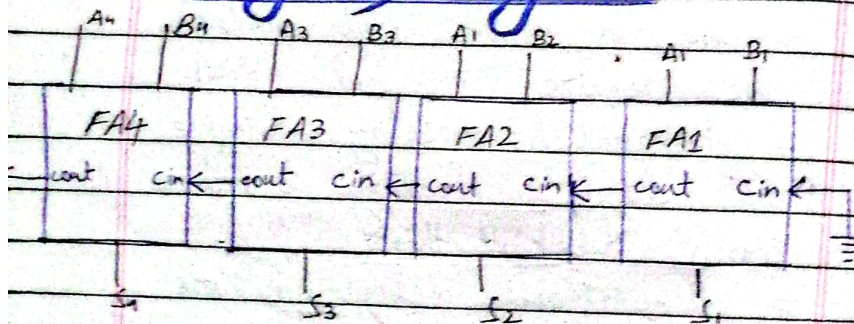
Date _____

Day MTWTF

⇒ Consider a NOT gate. The time taken for the NOT gate's output to become "0" after the application of logic "1" to the NOT gate's input is the propagation delay here.

⇒ Similarly the carry propagation delay is the time elapsed between the application of the carry in signal and the occurrence of the carry out (Cont) signal.

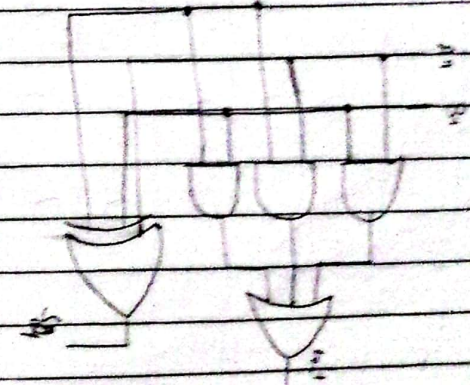
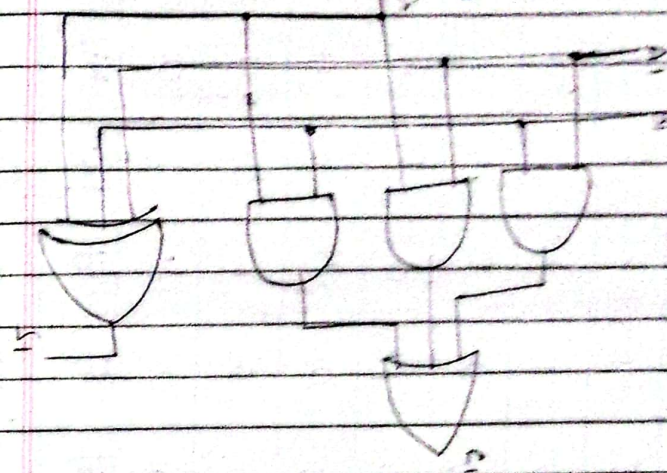
Ripple Carry Adder Logic Diagram



Date _____

Day MTWTF

Ripple Carry Adder Circuit Diagram



X

Day MTWTF

Date _____

-	-	-	-	-	0	0	A1
1	1	1	0	0	1	0	A2
1	1	0	1	0	0	0	A3
-	0	0	0	1	0	0	A4
1	1	1	1	1	0	0	B4
1	1	1	0	0	1	0	B3
-	-	0	1	0	0	0	B2
1	0	0	0	0	0	1	B1
1	1	1	0	0	1	0	S4
1	1	0	1	0	0	0	S3
1	0	0	0	0	0	0	S2
0	0	0	0	0	0	0	S1
1	1	1	1	1	0	0	Carry

Ripple Carry Adder Truth Table

Ripple Carry Adder Applications:

→ These carry adders are used mostly in addition to n-bit input sequences.

→ These carry adders are applicable in the digital signal processing and micro-processors.

Advantages:

→ we can perform addition process for n-bit sequences to get accurate results.

→ The designing of this adder is not a complex process.

Date _____

Day MTWTF

Look Ahead Carry

→ Invented by Weinberger.

Definition:

Carry Look Ahead adder is an improved version of the ripple carry adder.

→ It generates the carry-in of each full adder simultaneously without causing any delay.

→ The time complexity of carry look ahead adders = $O(\log n)$.

Carry Look Ahead Adder

Working :-

The working of carry look ahead adder is based on the principle:-

→ The carry-in of any stage full adder is independent of the carry bits generated during intermediate stage.

This adder reduces time the carry delay by reducing the number of gates through which a carry signal must propagate.

Date _____

Day MTWTF

The carry-in of any stage full adder depends only on the following two parameters:

→ Bits being added in the previous stages.

→ Carry-in provided in the beginning.

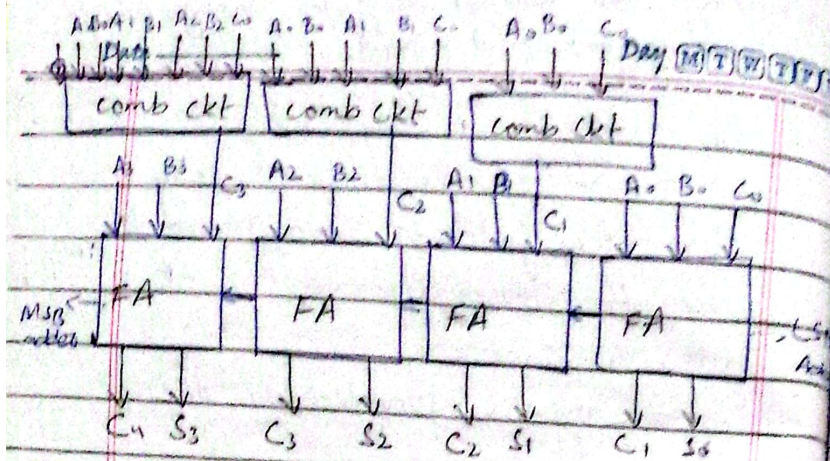
Now,

→ The above two parameters are always known from the beginning.

→ So, the carry-in of any stage full adder can be evaluated at any instant of time.

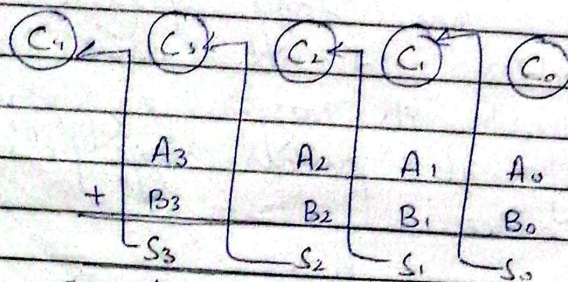
→ Thus, any full adder need not wait untill its carry-in is generated by its previous stage full adder.

Logic Diagram:



4-bit Look Ahead Adder

Consider two 4-bit numbers $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$ are to be added.



From here, we have -

$$C_1 = C_0(A_0 \oplus B_0) + A_0B_0$$

$$C_2 = C_1(A_1 \oplus B_1) + A_1B_1$$

$$C_3 = C_2(A_2 \oplus B_2) + A_2B_2$$

$$C_4 = C_3(A_3 \oplus B_3) + A_3B_3$$

for simplicity, let:-

$\rightarrow G_i = A_iB_i$ where G called carry generator

$\rightarrow P_i = A_i \oplus B_i$ where P is called carry propagator.

Then, rewriting above equation

$$C_1 = C_0P_0 + G_0 \quad \text{--- (i)}$$

$$C_2 = C_1P_1 + G_1 \quad \text{--- (ii)}$$

$$C_3 = C_2P_2 + G_2 \quad \text{--- (iii)}$$

$$C_4 = C_3P_3 + G_3 \quad \text{--- (iv)}$$

Now,

$\rightarrow$ Clearly C_1, C_2, C_3 are intermediate carry bits

$\rightarrow$ So, let's remove C_1, C_2 and C_3 from RHS of every equation.

$\rightarrow$ Substituting (i) in (ii)

we get C_2 in terms of C_0

$\rightarrow$ Then substituting (2) in

(3), we get C_3 in terms of C_0 and so on.

$$C_1 = C_0P_0 + G_0$$

$$C_2 = C_0P_0P_1 + G_0P_1 + G_1$$

$$C_3 = C_0P_0P_1P_2 + G_0P_1P_2 + G_1P_2 + G_2$$

$$C_4 = C_0P_0P_1P_2P_3 + G_0P_1P_2P_3 +$$

$$G_1P_2P_3 + G_2P_3 + G_3$$

Date _____

Day MTWTF

Trick to memorize Above Equation

As an example, consider the equation for generating carry bit C_2 .

There are three reasons for generation of C_2 as depicted in the following.

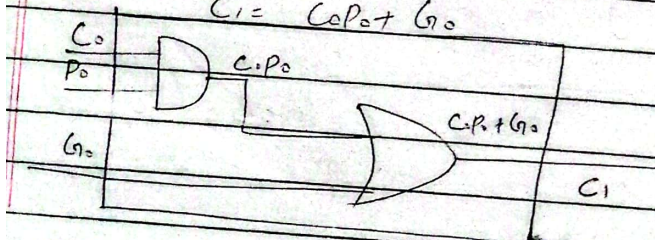
$$C_2 = (C_0 P_0 P_1) + (G_0 P_0) + (G_1) \rightarrow \text{it is generated in stage } S_1$$

Either it is C_0 & it is generated propagated through stages S_0 & S_1 .
 or it is generated in stage S_0 & then propagated through stage S_1 .

Implementation of C_1 :

→ It requires 1 AND gate & 1 OR gate.

$$C_1 = C_0 P_0 + G_0$$



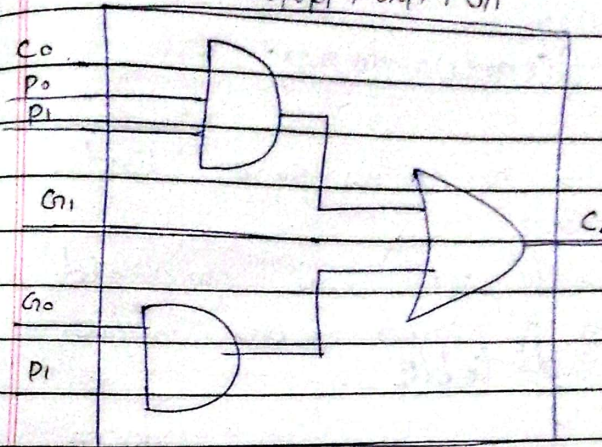
Date _____

Day MTWTF

Implementation of C_2

→ It requires 2 AND gates & 1 OR gate

$$C_2 = C_0 P_0 P_1 + G_0 P_1 + G_1$$



Implementation of C_3 & C_4 :

→ Use 3- AND gates & 1 OR gate

→ Use 4 AND gates & 1 OR gate.

General formula:- calculate number of gates required for evaluating all C_n .

- Number of AND gates = $n(n+1)/2$
- Number of OR gates = n

Date _____

Day MTWTFAdvantages:

→ It generates the carry look ahead in for each full adder simultaneously.

→ It reduces propagation delay.

Disadvantages:

→ It involves complex hardware.

→ It gets more complicated as the number of bits increases.

Truth table

A	B	C_i	C_{i+1}	Condition
0	0	0	0	No carry generate $C_{out}=0$
0	0	1	0	
0	1	0	0	No carry propagate $C_{out}=C_i$
0	1	1	1	
1	0	0	0	No carry propagate $C_{out}=C_i$
1	0	1	1	
1	1	0	1	Carry generate $C_{out}=1$
1	1	1	1	

$$S_i = P_i \oplus C_i \quad \therefore P_i = A_i \oplus B_i$$

$$C_{i+1} = G_i + P_i C_i \quad \therefore G_i = A_i B_i$$

Date _____

Day MTWTFCarry generation:

→ Carry generation occurs when an output carry is produced internally by the full adder.

→ A carry is generated only when both input bits are 1s.

→ The generated carry C_g is expressed as the AND function of the two input bits, A and B.

$$C_g = AB$$

Carry propagation:

→ Carry propagation occurs when the input carry is rippled to become output carry.

→ An input carry may be propagated by the full-adder when either or both of the input bits are 1s.

→ The propagated carry C_P is expressed as the OR function of the input bits.

$$C_P = A + B$$

Stage-1:-

$$\text{cout}_1 = C_{g1} + C_{p1} C_{in1}$$

$$\text{cin}_2 = \text{cout}_1$$

$$\text{cout}_1 = \text{cin}_2$$

Stage 2:-

$$\text{cout}_2 = C_{g2} + C_{p2} \text{cin}_2$$

$$\text{cout}_2 = C_{g2} + C_{p2} \text{cout}_1$$

$$= C_{g2} + C_{p2} (C_{g1} + C_{p1} C_{in1})$$

$$\text{cout}_2 = C_{g2} + C_{p2} C_{g1} + C_{p1} C_{p2} C_{in1}$$

Stage-3:-

$$\text{cin}_3 = \text{cout}_2$$

$$\text{cout}_3 = C_{g3} + C_{p3} \text{cin}_3$$

$$= C_{g3} + C_{p3} \text{cout}_2$$

$$= C_{g3} + C_{p3} (C_{g2} + C_{p2} C_{g1} + C_{p1} C_{p2} C_{in1})$$

$$= C_{g3} + C_{p3} C_{g2} + C_{p2} C_{p3} C_{g1} + C_{p1} C_{p2} C_{p3} C_{in1}$$

Date _____

Day **M** **T** **W** **T** **F** **S**

Stage - IV -

C_{in4} cont3

$$C_{out4} = C_{g4} + C_{p4} C_{out3}$$

$$= C_{g4} + C_{p4} (C_{g3} + C_{g2} C_{p3} +$$

$$C_{p2} C_{p3} C_{g1} + C_{p1} C_{p2} C_{p3} C_{in1})$$

$$= C_{g4} + C_{g3} C_{p4} + C_{g2} C_{p3} C_{p4}$$

$$+ C_{p4} C_{p2} C_{p3} C_{g1} + C_{p1} C_{p2} C_{p3} C_{p4} C_{in1}$$

Date _____

Day MTWTFSS

Q# What is difference b/w
Half adder and full
adder?

→ Half adder

Half adder is a combinational logic circuit which adds two 1-bit digits. The half adder produces a sum of the two inputs.

→ Previous carry is not used.

→ In Half adder there are two input bits (A, B)

→ Logical expression for half adder is:

$$S = A \oplus B$$

$$C_{out} = A \cdot B$$

→ It consists of one XOR gate and one AND gate.

Full adder

Full adder is a combinational logic circuit that performs an addition operation on three one-bit binary numbers.

The full adder

produces the sum of the three inputs and carry value.

→ Previous carry is used.

→ In full adder there are three input bits (A, B, C_{in}).

→ Logical expression for full adder is:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + (C_{in}(A \oplus B))$$

→ It is used in Calculators, Computers, digital measuring devices etc.

→ It consists of two XOR gate and two AND gate and one OR gate.

It is use in multiple bit addition and digital processors etc.

Q# What is the major limitation of half adder?

Half adders have a major limitation is that they cannot accept a carry bit from a previous stage, meaning that they cannot be chained together to add multi-bit.

Q# How to design Combinational circuit?

Steps: 1) Problem definition

- 2) The no of available input variables and
- 3) required o/p variables is determined.
- 4) The i/p and O/P variables are assigned letter symbols.
- 5) make a truth table to define the relationship b/w i/p and o/p.
- 6) Get simplified boolean function for each o/p is obtained.

then the logic diagram